

Simulación de Pruebas de Penetración Web con Burp Suite y DVWA en Kali Linux

Este documento presenta el desarrollo técnico del Ejercicio Práctico 2, que tiene como propósito realizar pruebas de penetración básicas en una aplicación vulnerable utilizando Burp Suite y DVWA desde un entorno controlado en Kali Linux. Se busca comprender la interceptación y manipulación de solicitudes HTTP y validar una posible vulnerabilidad de inyección SQL.

Etapas 1: Configuración del Entorno de Pruebas

- **Sistema Operativo:** Kali Linux
- **Aplicación vulnerable:** DVWA (Damn Vulnerable Web Application)
- **URL local:** <http://localhost/dvwa>
- **Proxy interceptante:** Burp Suite configurado en 127.0.0.1:8080
- **Navegador utilizado:** Firefox con configuración de proxy manual

Procedimiento:

Se iniciaron los servicios web requeridos:

```
sudo service apache2 start
```

```
sudo service mysql start
```

- 1.
2. DVWA fue accedido exitosamente desde el navegador.
3. Burp Suite fue abierto y configurado para interceptar el tráfico web.
4. Firefox fue ajustado para enviar todo el tráfico HTTP a través del proxy local en el puerto 8080.

Etapla 2: Interceptación de la Solicitud HTTP

1. Se accedió al módulo vulnerable "SQL Injection" en DVWA.
2. En el campo "User ID" se ingresó el valor 1 y se presionó "Submit".

Burp Suite interceptó la solicitud HTTP:

GET /dvwa/vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1

- 3.
4. Se confirmó que la petición fue recibida correctamente y quedó disponible para su edición.

Etapla 3: Manipulación del Parámetro Vulnerable

Se modificó el valor del parámetro `id` por el siguiente payload de prueba:

`id=1' OR '1'='1`

- 1.

La solicitud quedó codificada como:

GET /dvwa/vulnerabilities/sqli/?id=1'%20OR%20'1'%3D'1&Submit=Submit HTTP/1.1

- 2.
3. Se reenvió la petición modificada al servidor.

Resultado Observado:

- La aplicación devolvió un listado de múltiples usuarios, incluso sin autenticación previa.

Etapa 4: Documentación de Resultados

Parámetro Modificado:

id=1' OR '1'='1

-
- **Resultado:**
La respuesta del servidor mostró el contenido completo de la tabla `users`, lo que indica que se alteró la consulta SQL original.
- **Interpretación:**
La entrada del usuario no fue correctamente validada ni saneada. El payload permitió alterar la lógica de autenticación y obtener información que debería estar restringida.

Etapa 5: Recomendación Técnica para Mitigación

Para mitigar este tipo de vulnerabilidad, se deben implementar las siguientes medidas:

1. Utilizar **consultas preparadas (prepared statements)** en el código del backend para separar lógica de negocio de los datos ingresados por el usuario.
2. Validar y sanear todas las entradas provenientes del cliente mediante filtrado estricto del tipo y longitud de los datos.
3. Evitar la **concatenación directa** de variables en las consultas SQL.
4. Implementar módulos de detección de intrusos (WAF o IDS) para monitorear comportamiento anómalo.

Análisis Reflexivo

La realización de esta prueba permitió observar de forma directa cómo una aplicación vulnerable puede ser manipulada a partir de una solicitud HTTP básica. La alteración del parámetro `id` utilizando un payload de SQLi permitió eludir validaciones y obtener información confidencial. Este ejercicio evidencia la importancia de aplicar principios de desarrollo seguro, validar estrictamente los datos de entrada y realizar auditorías frecuentes en ambientes controlados y autorizados.